

Lucrare practică nr. 5

Tema: Rețele adversariale generative (GAN)

Scopul lucrării: Implementarea unei rețele GAN în Python

Obiectivele lucrării:

1. Înțelegerea conceptului de IA generativă. Studii de caz.
2. Crearea modelului, antrenarea și optimizarea rețelei.
3. Evaluarea rezultatelor.

1.1 Noțiuni de bază:

În această lucrare se va analiza arhitectura și funcționarea rețelei GAN. Din aplicațiile practice ce folosesc GAN: generare imagini, augmentare date, sinteză text-imagine, generare text, detectare conținut (text, imagini) real/generat va fi analizat un exemplu de clasificare a imaginilor reale și cele generate.

Ideea principală a modelelor generative este de a explora funcția de verosimilitate și estimarea probabilității pentru a modela date noi. În cazul modelelor generative pentru a găsi probabilitatea, este crucială estimarea probabilității a priori $P(Y)$ și a probabilității de verosimilitate $P(X|Y)$ cu ajutorul datelor de antrenament, iar utilizând Teorema Bayes, este posibil să se calculeze probabilitatea posterioară $P(Y|X)$.

Diferența cheie cu modelele discriminative constă în posibilitatea de a genera noi date. O rețea generativă adversarială conține două rețele neuronale: un **generator** – rețea neuronală convoluțională care pe baza unui semnal aleatoriu generează date similare cu cele reale din setul de date și un **discriminator** – care clasifică/diferențiază datele reale vs. construite/generate.

Termenul adversarial provine de la *adversari* : generator vs. discriminator. Cele două rețele, sunt antrenate simultan astfel încât generatorul să maximizeze probabilitatea ca discriminatorul să nu recunoască datele generate, iar discriminatorul are ca scop să minimizeze această probabilitate. Astfel ambele rețele se îmbunătățesc în timp.

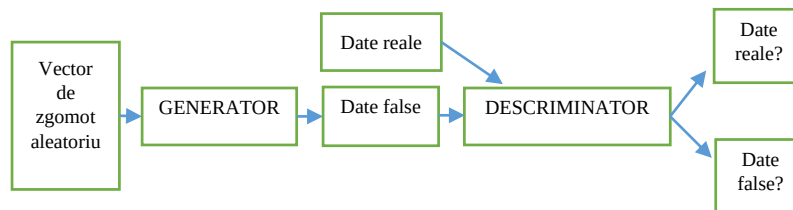


Figura 9.1 Arhitectura rețelei GAN

Pașii GAN:

1. **Inițializare:** Citirea și preprocesarea setului de date. Crearea vectorilor de zgomot aleatoriu ca date de intrare pentru generator. Crearea rețelelor generator și discriminator.
2. **Generare:** Generatorul procesează zgomotul aleatoriu, creează caracteristici pentru a produce date similare cu cele reale.
3. **Discriminare:** Discriminatorul evaluează datele (generate + reale) și prezice dacă sunt reale sau false.
4. **Ciclu de feedback:** Erorile discriminatorului sunt folosite pentru a ajusta ponderile ambelor rețele.

5. **Antrenament** : Se stabilește un număr de epoci, ambele rețele se îmbunătățesc continuu, generatorul tinde să producă date pe care discriminatorul să nu le mai poată distinge de cele reale.

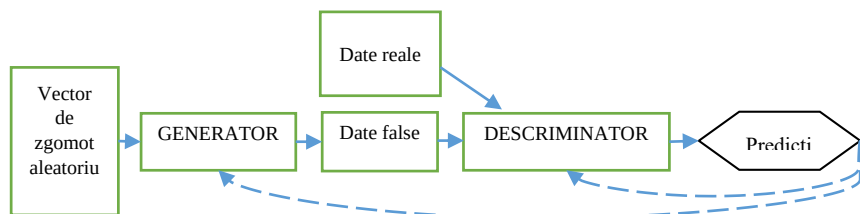


Figura 9.2 Antrenarea rețelei adversariale generative

Cele mai multe exemple de aplicare a rețelelor adversariale generative sunt pe imagini.

Se va analiza un exemplu de rețea convoluțională generativă **DCGAN – Deep Convolutional GAN** [23].

Se va folosi <https://colab.research.google.com/> pentru optimizarea timpului de lucru.

Pasul 1. Importarea librărilor

```
!pip install tensorflow==2.12.0
import matplotlib.pyplot as plt
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers
```

Pasul 2. Încărcarea și preprocesarea setului de date

```
(X_train, y_train), (_, _) = tf.keras.datasets.mnist.load_data()
X_train.shape
Out[ ]
(60000, 28, 28)
```

```
i = np.random.randint(0, 60000)
#print(i)
print(y_train[i])
plt.imshow(X_train[i], cmap='gray');
```

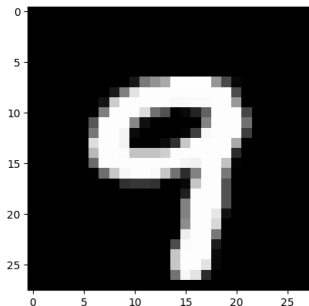


Figura 9.3 O imagine (reală) din setul de date

```
X_train = X_train.reshape(X_train.shape[0], 28, 28,
                          1).astype('float32')
X_train.shape
Out[ ]
(60000, 28, 28, 1)

#datele fiind imagini gri au valori detions la #[0:255]
X_train[0].min(), X_train[0].max()
Out[ ]
(0.0, 255.0)

#reprezentarea datelor pe intervalul [-1 : 1]
X_train = (X_train - 127.5) / 127.5
X_train[0].min(), X_train[0].max()
Out[ ]
(-1.0, 1.0)

buffer_size = 60000
batch_size = 256
buffer_size / batch_size # mini-batch
Out[ ]
234.375

type(X_train)
Out[ ]
numpy.ndarray

#transformăm datele de intrare în tensori
X_train =
    tf.data.Dataset.from_tensor_slices(X_train).shuffle(buffer_size).ba
    tch(batch_size)

type(X_train)
Out[ ]
tensorflow.python.data.ops.batch_op._BatchDataset
def __init__(input_dataset, batch_size, drop_remainder, name=None)
A `Dataset` that batches contiguous elements from its input.

X_train
Out[ ]
< BatchDataset element_spec=TensorSpec(shape=(None, 28, 28, 1),
dtype=tf.float32, name=None)>
```

Pasul 3. Crearea modelului generator

În Figura 9.4 se observă numărul de straturi ale rețelei convoluționale care are rolul de transformare a zgomotului în imagini gri ce ar reprezenta scrisul cifrelor de mână (similar cu cele

din setul de date MNIST). Generatorul folosește `tf.keras.layers.Conv2DTranspose` pentru crearea de caracteristici [24].

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 12544)	1254400
batch_normalization (Batch Normalization)	(None, 12544)	50176
leaky_re_lu (LeakyReLU)	(None, 12544)	0
reshape (Reshape)	(None, 7, 7, 256)	0
conv2d_transpose (Conv2DTranspose)	(None, 7, 7, 128)	819200
batch_normalization_1 (Batch Normalization)	(None, 7, 7, 128)	512
leaky_re_lu_1 (LeakyReLU)	(None, 7, 7, 128)	0
conv2d_transpose_1 (Conv2DTranspose)	(None, 14, 14, 64)	204800
batch_normalization_2 (Batch Normalization)	(None, 14, 14, 64)	256
leaky_re_lu_2 (LeakyReLU)	(None, 14, 14, 64)	0
conv2d_transpose_2 (Conv2DTranspose)	(None, 28, 28, 1)	1601
Total params: 2,330,945		
Trainable params: 2,305,473		
Non-trainable params: 25,472		

Figura 9.4 Modelul generatorului

```
generator.input
Out[ ]
<KerasTensor: shape=(None, 100) dtype=float32 (created by layer
'dense_input')>
```

```
noise = tf.random.normal([1, 100])
noise
Out[ ]
```

```
<tf.Tensor: shape=(1, 100), dtype=float32, numpy=
array([[ 0.36760178, -0.5585843, -0.99277544, -0.30362052,  1.9594874,
        -0.7995758, -1.0425128, -1.9353644,  0.46869054, -1.0198084,
        -0.06606854,  1.6819725,  1.38074,  0.23095237,  0.06632625,
         0.02222416, -0.4786155, -0.28813082, -0.00673659,  0.9135299,
        -1.8283585, -0.11663245,  1.640456,  0.13198088, -0.20053442,
        -0.7259959,  0.27556407,  1.7568831,  0.60040927, -0.624015,
         0.15185314,  0.8846243,  1.4947522,  0.9872027, -1.4122751,
        -1.0147512, -0.1397007,  1.009136, -2.5516677,  0.25509268,
        -0.7045359, -1.3148022,  1.7150024,  0.2943509, -0.879051,
        -0.8772213,  1.382248, -0.64826316, -1.2990009,  0.7134725,
         0.8366656, -0.81093544,  1.5425649, -0.9021319, -0.99275446,
         1.0897568,  1.2420532,  0.01360834,  0.5868647,  0.5968689,
         0.7338235,  0.4924595,  0.614351, -1.2298661, -0.7311326,
        -0.67100847, -1.1116968,  0.02627075,  1.5599171, -2.2266283,
        -1.1961274, -0.7356992,  2.2358198, -1.2968272,  1.1968703,
         1.3603082, -0.03814069,  0.2707285, -0.33872154,  0.64347726,
         0.25967428, -1.1865146,  0.40149105,  0.20739253,  0.8993141,
         0.83216196, -0.41801596,  0.39632636, -0.5793846, -0.8568164,
         0.39943206, -0.7910329,  0.5574451, -0.64192766, -1.3065021,
         0.9738948,  1.3017397, -0.7225262, -0.10140286, -0.29319495]],
        dtype=float32)>
```

Figura 9.5 Vectorul de zgomot

```
generated_image = generator(noise, training = False)
generated_image.shape
Out[ ]
TensorShape([1, 28, 28, 1])
```

E important ca datele de intrare să fie transformate în tensori pentru a putea fi procesate de rețelele neuronale (generator și discriminator).

Vizualizarea unui vector de zgomot ce reprezintă datele de intrare a generatorului:

```
plt.imshow(generated_image[0, :, :, 0], cmap='gray');
```

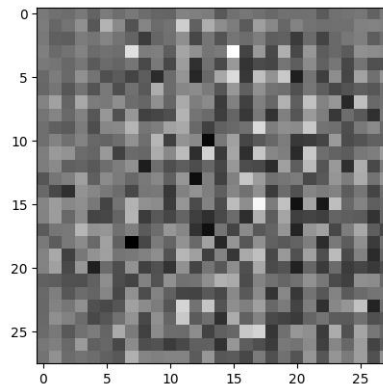


Figura 9.6 Zgomot aleatoriu ca intrare pentru generator

Pasul 4. Crearea modelului discriminator

Discriminatorul este o rețea neuronală convoluțională pentru clasificare binară.

Model: "sequential_1"		
Layer (type)	Output Shape	Param #
=====		
conv2d (Conv2D)	(None, 14, 14, 64)	1664
leaky_re_lu_3 (LeakyReLU)	(None, 14, 14, 64)	0
dropout (Dropout)	(None, 14, 14, 64)	0
conv2d_1 (Conv2D)	(None, 7, 7, 128)	204928
leaky_re_lu_4 (LeakyReLU)	(None, 7, 7, 128)	0
dropout_1 (Dropout)	(None, 7, 7, 128)	0
flatten (Flatten)	(None, 6272)	0
dense_1 (Dense)	(None, 1)	6273
=====		
Total params: 212,865		
Trainable params: 212,865		
Non-trainable params: 0		

Figura 9.7 Modelul discriminator

```
discriminator.input
Out[ ]
<KerasTensor: shape=(None, 28, 28, 1) dtype=float32 (created by layer
'conv2d_input')>
```

```
discriminator(generated_image, training = False) # logits
Out[ ]
<tf.Tensor: shape=(1, 1), dtype=float32, numpy=array([[0.00227323]],
dtype=float32)>
```

```
tf.sigmoid(-0.00081761)
Out[ ]
<tf.Tensor: shape=(), dtype=float32, numpy=0.4997956>
```

Pasul 5. Calcularea funcțiilor de pierdere pentru generator și discriminator.

Compararea predicțiilor discriminatorului asupra datele reale cu o matrice de 1 și predicțiile discriminatorului asupra imaginilor false (generate) cu o matrice de 0 simplifică determinarea erorilor.

```
cross_entropy = tf.keras.losses.BinaryCrossentropy(from_logits = True)

tf.ones_like(1)
Out[ ]
<tf.Tensor: shape=(), dtype=int32, numpy=1>
```

```

def discriminator_loss(expected_output, fake_output):
    real_loss = cross_entropy(tf.ones_like(expected_output),
                               expected_output)
    fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
    total_loss = real_loss + fake_loss
    return total_loss

def generator_loss(fake_output):
    return cross_entropy(tf.ones_like(fake_output), fake_output)

generator_optimizer = tf.keras.optimizers.Adam(learning_rate =
0.00001)
discriminator_optimizer = tf.keras.optimizers.Adam(learning_rate =
0.00001)

```

Pasul 6. Antrenarea modelului GAN și vizualizarea rezultatelor.

```

X_train
Out[ ]
<_BatchDataset element_spec=TensorSpec(shape=(None, 28, 28, 1),
dtype=tf.float32, name=None)>

```

```

epochs = 100
noise_dimension = 100
number_of_images = 16

```

```

batch_size, noise_dimension
Out[ ]
(256, 100)

```

```

@tf.function #consultați [23, 25]
...

```

```

def train_gan(dataset, epochs, test_images):
    for epoch in range(epochs):
        #print(epoch)
        for image_batch in dataset:
            #print(image_batch.shape)
            train(image_batch)

    print('Epoch: ', epoch + 1)
    generated_images = generator(test_images, training = False)
    fig = plt.figure(figsize = (10,10))
    for i in range(generated_images.shape[0]):

```

```
plt.subplot(4,4,i + 1)
plt.imshow(generated_images[i, :, :, 0] * 127.5 + 127.5,
cmap='gray')
plt.axis('off')
plt.show()
```

```
train_gan(X_train, epochs, test_images)
```

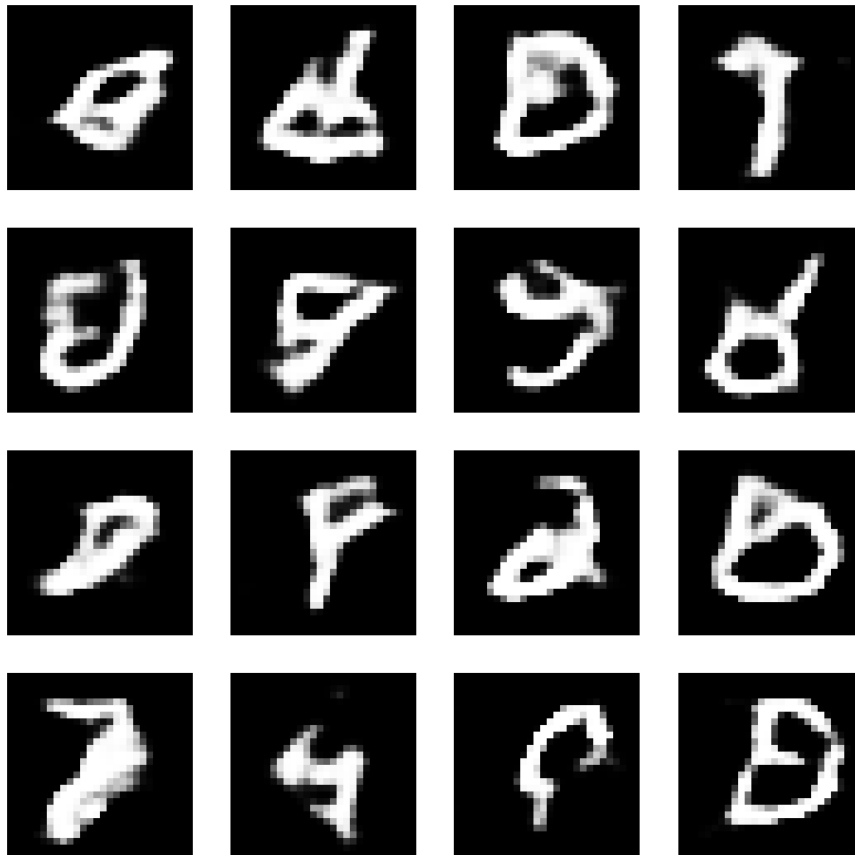


Figura 9.8 Imagini generate la 100 de epoci de antrenare

9.2 Sarcina individuală:

1. Alegeți o problemă care să permită implementarea unui model GAN.
2. Creați modelul rețelei adversariale generative. Descrieți pașii și includeți comentarii în cod pentru o mai bună înțelegere a informației.
3. Calculați pierderile (funcțiile generator loss și discriminator loss), optimizați.
4. Scrieți concluzii.

9.3 Evaluare și livrabile

Studentii vor demonstra cunoștințele și abilitățile dobândite ca urmare a realizării lucrării practice. Vor explica pașii creării unei rețele adversariale generative, vor face comparație între Conv2DTranspose și Conv2D, vor da exemple de aplicare a modelului GAN.

Studentii vor prezenta un fișier cu extensia .py și un raport .doc care conține: tema, obiectivele, sarcina, codul sursă și capturi de ecran cu rezultatele obținute la rularea codului, neapărat o concluzie despre ce au reușit să asimileze efectuând sarcinile din lucrare.

Modelul de raport poate fi consultat în Anexa 1.

Întrebări de autoevaluare:

1. Care sunt pașii de bază pentru crearea unei arhitecturi GAN?
2. Cum determinăm câte straturi vor fi pentru generator și discriminator?
3. Care sunt datele de intrare pentru generator și discriminator?
4. Care e utilitatea Conv2DTranspose și Conv2D?
5. Cum se calculează funcția `loss` pentru generator și discriminator?
6. Cum are loc antrenarea rețelei GAN?
7. Cum e posibilă îmbunătățirea rezultatelor generatorului?
8. Ce funcții de activare folosim în straturile interne, dar în stratul de ieșire?
9. Dați exemple de arhitecturi GAN, care sunt avantajele fiecărui tip?
10. Dați exemple de aplicare a modelului GAN.